

* Title: Write a c++ program to perform the following operations.

1. Insert an element into a binary search tree.
2. Delete an element from a binary search tree.
3. Search for a key element in a binary search tree.

* Aim:- Write a c++ program to implement Binary Search tree (BST) including all operations:

Q.1) What is Binary Search trees algorithm?

Sol:- A Binary Search Tree (BST) is a hierarchical data structure that organizes data in a way that allows for efficient searching, insertion, and deletion operations. It is a specialized type of binary tree with the following key properties: Each node in a BST can have at most two children: a left child and a right child.

1. Insert

Insert operation adds a new node in a binary search tree.

Algorithm:-

Insert(data)

Begin

If node == null

Return createNode(data)

If (data > root->data)

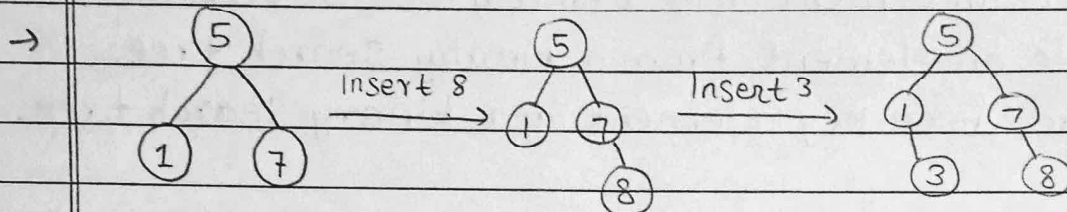
Node->right = insert(Node->right, data)

Else If (data < root->data)

Node->left = insert(Node->left, data)

Return Node;

end

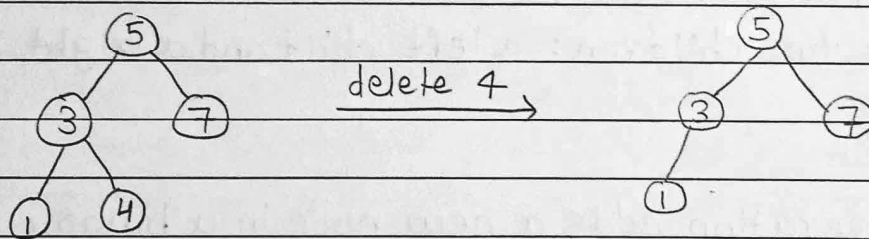


2. Delete.

Delete operation deletes a node that matches the given Key from BST.

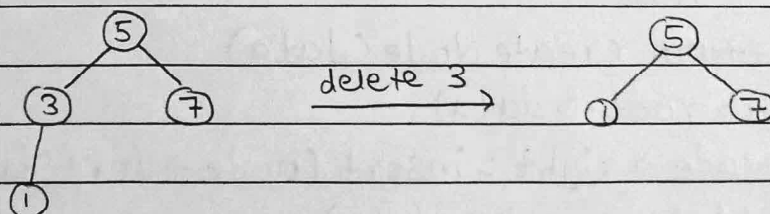
i. When the node is Leaf node.

When the node to be deleted is leaf node, then we directly delete the Node.



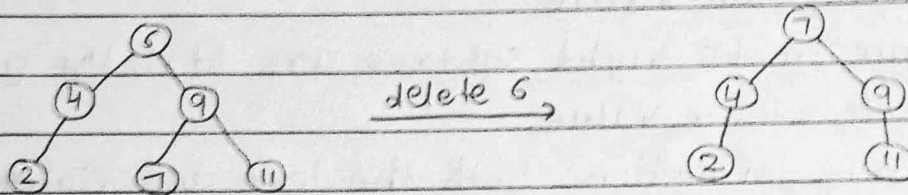
ii. When the node has only one child.

When the node to be deleted has only one child, then we copy into the node and delete the child.



(iii) When the node has two children

If the node to be deleted has two children, then we find the inorder successor for the node and then copy the inorder successor to the node. Later, we delete the inorder successor.



3. Search.

The Search operation of BST searches for a particular item identified as "key" in BST.

algorithm:-

Search (Key)

Begin

If (root == null || root → data = Key)

Return Root;

If (root → Key < Key)

Search (root → left, Key)

Else if (root → Key > Key)

Search (root → right, Key)

end

Q.2) How does binary search trees work?

Solⁿ:- core principles:

- Binary Structure: Each node in a BST can have at most two child nodes: a left child and a right child.
- ordering property: for any given node
 - All value in its left subtree are strictly less than the node's own value.
 - All value in its right subtree are strictly greater than the node's own value.
- Recursive definition: Both the left and right subtree of any node must also adhere to the binary search tree properties.

How binary search trees work:

- Searching: To find a value, start at the Node root. If the target value matches the current node's value, move to the left child. If it's greater, move to the right child. This process repeats recursively until the value is found or a null pointer is encountered (indicating the value is not in the tree).
- Insertion: To insert a new value, follow the same search logic to find the correct position where the new node should be placed as a leaf. If the new value is less than the current node, attempt to insert it in the left subtree; otherwise, in the right subtree. When a null pointer is reached, create a new node at that position.

• Deletion: Deleting a node is more complex and depends on the node's children:

- No children: (leaf node): - Simply remove the node.
- one child: Replace the node with its child.
- Two child: find the in-order successor.

Q.3) What are the different applications of Binary Search tree?

Solⁿ: Binary Search trees (BSTs) are used in a wide variety of applications, including database and file systems indexing, implementing dictionaries and symbol table for compilers, and creating structures for auto completion and spell-checking. They also find use in network routing, priority queues, data compression, and certain game AI and simulations.

• Software and applications.

- Autocomplete and spell checker: BSTs can store dictionaries for fast lookups, enabling features like autocomplete and suggesting corrections for misspelled words.
- compilers: They are used to manage symbol tables, which store information about variables, functions, and keywords.
- Expression evaluation: BSTs can be used to parse and evaluate arithmetic expressions.

Networking and AI:

- Network routing: They can be used to implement routing tables, helping routers make efficient decisions.

- Game development: BSTs, particularly in the form of decision trees, are used in game AI to represent and evaluate possible actions and outcomes.

Q.4) Write the source code for Binary Search tree?

Solⁿ:-

```
#include <iostream>
using namespace std;

struct bstnode
{
    int data;
    struct bstnode *left, *right;
};

struct bstnode *newNode(int key)
{
    struct bstnode *temp = new struct bstnode();
    temp->data = key;
    temp->left = temp->right = NULL;
    return temp;
}

void inorder(struct bstnode *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout << root->data << " ";
    }
}
```

```

    inorder (root->right);
}
}

```

```

Struct bstnode *insert (Struct bstnode *node, int Key)
{
    if (node == NULL) return newNode (Key);
    if (Key < node->data)
        node->left = insert (node->left, Key);
    else
        node->right = insert (node->right, Key);

    return node;
}

```

```

Struct bstnode * minValueNode (Struct bstnode * node)
{
    Struct bstnode * current = node;
    while (current->left != NULL)
        current = current->left;

    return current;
}

```

```

Struct bstnode * deleteNode (Struct bstnode * root,
                              int Key)
{
    if (root == NULL)
        return root;
    if (Key < root->data)
        root->left = deleteNode (root->left, Key);
    else if (Key > root->data)

```



```
root->right = deleteNode (root->right, Key);
```

```
else
```

```
{
```

```
if (root->left == NULL)
```

```
{
```

```
struct bstnode *temp = root->right;
```

```
free(root);
```

```
return temp;
```

```
}
```

```
else if (root->right == NULL)
```

```
{
```

```
struct bstnode *temp = root->left;
```

```
free (root);
```

```
return temp;
```

```
}
```

```
struct bstnode *temp = minValueNode (root->right);
```

```
root->data = temp->data;
```

```
root->right = deleteNode (root->right, temp->data);
```

```
}
```

```
return root;
```

```
}
```

```
// main program.
```

```
int main()
```

```
{
```

```
struct bstnode *root = NULL;
```

```
root = insert (root, 40);
```

```
root = insert (root, 30);
```

```
root = insert (root, 60);
```

```
root = insert (root, 65);
```

```
root = insert (root, 70);
```

```

cout << "Binary Search Tree created (Inorder
traversal): " << endl;
inorder(root);
cout << "In Delete Node 40\n";
root = deleteNode (root, 40);
cout << "Inorder traversal for the modified Binary
Search Tree:" << endl;
inorder(root);
return 0;
}

```

output:

Binary Search tree created (Inorder Traversal):
30 40 60 65 70

Delete Node 40

Inorder Traversal for the modified Binary Search tree:
30 60 65 70

Q.5) Which rule is used to perform the Binary Search tree algorithm?

Solⁿ:- The core rule governing the structure and operations within a Binary Search Tree (BST) is as follows:

For any given node in a Binary Search Tree:

- left Subtree Rule: All values in the left child and its entire subtree (including all descendants) must be strictly less than the value of the current Node.

- **Right Subtree Rule:** All values in the right child and its entire subtree (including all descendants) must be strictly greater than the value of the current Node.

This fundamental rule ensures the ordered property of a BST, which is crucial for efficient searching, insertion, and deletion operations. When performing a search, for instance, this rule allows for the elimination of half of the remaining tree at each step, similar to a binary search in a sorted array.